Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и ОА в ИЗ»
на тему «Бинарное дерево поиска»

Выполнили:
студенты группы 25ВВВ1
Новиков И.А.
Евстратов В.С.

Принял:
к.т.н., доцент Юрова О.В.

Пенза 2026

Цель работы

Изучение нелинейных динамических структур данных языка Си на примере бинарного дерева поиска, освоение рекурсивных алгоритмов обработки деревьев (создание, вывод, поиск, подсчёт вхождений), работа с указателями и динамической памятью (malloc, free), а также оценка сложности процедуры поиска по значению в бинарном дереве.

Лабораторное задание

На основе приведённого в методических указаниях кода бинарного дерева поиска (функции CreateTree и print_tree) выполнить следующие задания:

Задание 1: Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.

Задание 2: Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Задание 3 (*): Изменить функцию добавления элементов для исключения добавления одинаковых элементов.

Задание 4 (*): Оценить сложность процедуры поиска по значению в бинарном дереве.

Описание метода решения задач

Во всех заданиях используется одно и то же бинарное дерево поиска. Узел дерева описывается структурой, в которой хранятся целое число и указатели на левого и правого потомков:

```
struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};
```

Обращение к дереву осуществляется через глобальный указатель root. Каждое задание перед началом работы обнуляет этот указатель, поэтому деревья разных режимов не влияют друг на друга. Память под узел выделяется функцией malloc (с проверкой результата), освобождается функцией free.

Порядок размещения значений определяется исходной функцией CreateTree: значение, строго большее значения узла, помещается в левое

поддерево, а меньшее или равное – в правое. Поэтому при выводе функцией print_tree (дерево повёрнуто на 90 градусов, корень слева, правое поддерево выводится первым) меньшие значения оказываются сверху, большие – снизу. Корень находится на уровне 0, потомки корня – на уровне 1 и так далее.

Задание 1 (поиск, функция search_level): функция рекурсивная и принимает указатель на текущий узел, искомое значение и номер текущего уровня. Если указатель равен NULL, значение в дереве отсутствует, возвращается -1. Если значение равно данным узла, возвращается номер уровня. Иначе поиск продолжается в левом поддереве (если искомое значение больше значения узла) или в правом (в остальных случаях) с увеличением уровня на единицу. Благодаря упорядоченности дерева на каждом шаге отбрасывается одно из поддеревьев, то есть просматривается только один путь от корня. Для найденного значения выводится его уровень, иначе – сообщение «Not found». Если значение встречается несколько раз, возвращается уровень ближайшего к корню элемента.

Задание 2 (подсчёт вхождений, функция count_val): при добавлении равное значение всегда направляется в правое поддерево узла, поэтому все вхождения искомого числа лежат на одном пути от корня. Функция спускается по этому пути так же, как поиск; при совпадении она выводит уровень найденного элемента, увеличивает счётчик (передаётся по указателю) и продолжает поиск в правом поддереве найденного узла. По достижении NULL обход заканчивается, а значение счётчика равно числу вхождений.

Задание 3 (дерево без дубликатов, функция CreateTreeUnique): функция повторяет CreateTree, но при спуске по дереву дополнительно сравнивает добавляемое значение со значением текущего узла. Если они равны, выводится сообщение «Duplicate N, skipped» и функция возвращает корень без создания узла. Память под узел выделяется только после того, как найдено свободное место, поэтому отклонённый дубликат не приводит к утечке памяти. В этом режиме подсчёт вхождений может давать только 0 или 1.

Задание 4 (оценка сложности): при поиске на каждом уровне выполняется конечное число операций (одно-два сравнения и переход к потомку), а количество просмотренных уровней не превышает h + 1, где h –

высота дерева. Следовательно, сложность поиска по значению равна O(h), и она зависит от формы дерева:

Случай	**Форма дерева**	**Высота h**	**Сложность поиска**
Лучший	искомое значение в корне	0	O(1)
Средний	сбалансированное или построенное из случайных чисел	порядка log2 n	O(log n)
Худший	вырожденное (числа вводились по возрастанию или убыванию)	n − 1	O(n)

В худшем случае дерево превращается в односвязный список, и поиск сводится к линейному просмотру. Подсчёт вхождений и добавление элемента также выполняют один спуск по дереву, поэтому имеют ту же сложность O(h); глубина рекурсии при этом также не превышает h + 1.

Дополнительная функция (invert_level): в программу включена функция инверсии уровня. Она рекурсивно спускается до заданного уровня и у всех узлов этого уровня меняет местами левое и правое поддеревья. После инверсии упорядоченность поддеревьев нарушается, поэтому поиск и подсчёт следует выполнять до неё.

Пояснительный текст к программе

Структура программы: при запуске выводится меню выбора режима (1 – дерево с дубликатами, 2 – дерево без дубликатов), после чего вызывается функция task1 или task2. Каждая из них содержит цикл с меню операций: 1 – добавить число, 2 – вывести дерево, 3 – поиск значения, 4 – подсчёт вхождений, 5 – инверсия уровня, 6 – оценка сложности, 7 – выход. Режимы отличаются только функцией добавления: task1 использует CreateTree, task2 – CreateTreeUnique. При выходе дерево полностью освобождается функцией free_tree.

Основные функции программы: CreateTree добавляет число в дерево (с дубликатами); CreateTreeUnique добавляет число, пропуская уже имеющиеся; print_tree выводит дерево, повёрнутое на 90 градусов, с отступом, пропорциональным уровню; search_level ищет значение и возвращает номер уровня; count_val подсчитывает число вхождений и выводит уровни найденных элементов; invert_level меняет местами

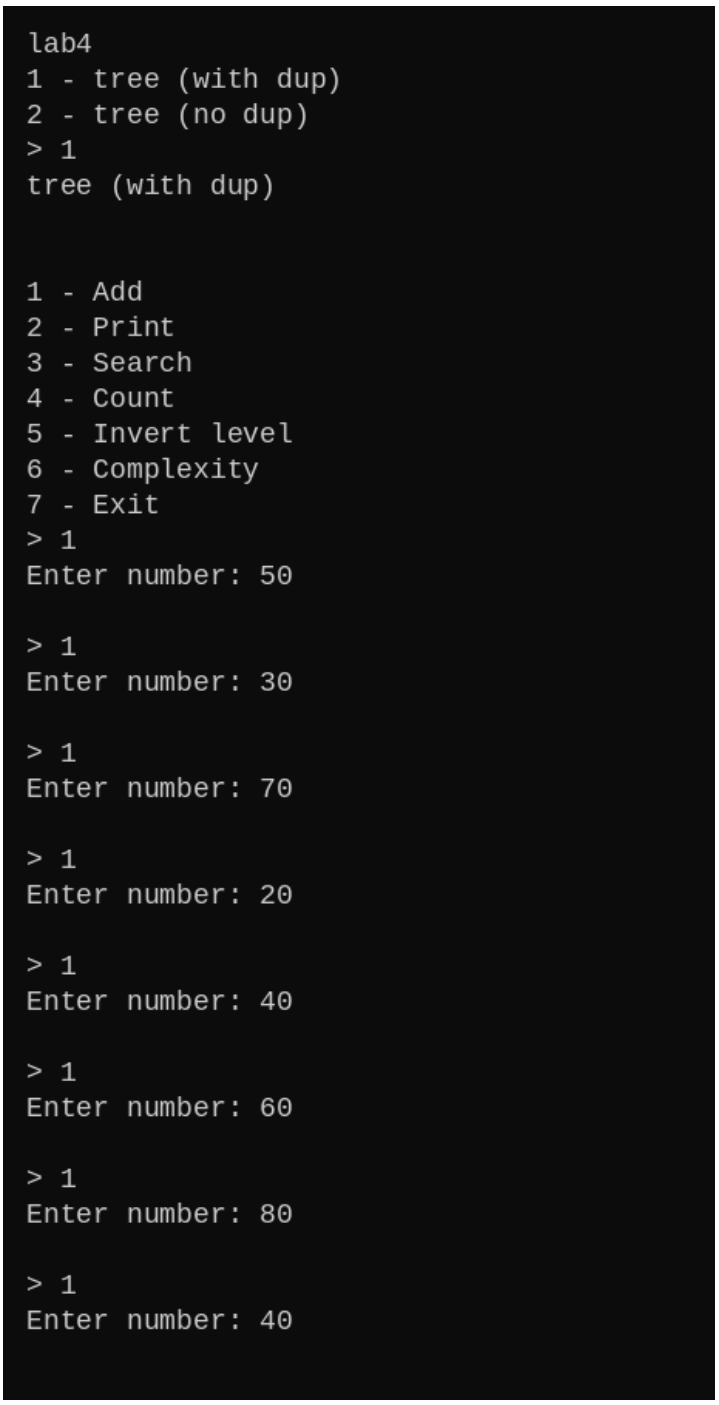
поддеревья у узлов заданного уровня; free_tree рекурсивно освобождает память всех узлов (сначала потомки, затем сам узел); task1 и task2 содержат меню и основной цикл работы.

Динамическое управление памятью: узлы создаются функцией malloc с проверкой результата на NULL (при ошибке выводится «Memory error» и программа завершается) и освобождаются функцией free_tree при выходе из режима. Это исключает утечки памяти.

Обработка граничных случаев: при выводе пустого дерева выдаётся сообщение «Tree is empty»; поиск в пустом дереве и поиск отсутствующего значения дают «Not found», подсчёт вхождений в этих случаях возвращает 0; повторное число в режиме без дубликатов отклоняется с сообщением «Duplicate N, skipped». Полный текст программы приведён в приложении.

Результаты работы программы

Дерево с дубликатами (режим 1). Последовательно добавлены числа 50, 30, 70, 20, 40, 60, 80 и повторно 40. Корень дерева – 50; меньшие значения выведены выше корня, большие – ниже. Повторное число 40 помещено в правое поддерево первого узла 40.



Далее выполнен вывод дерева, поиск значений 60 (найдено на уровне 2) и 99 (не найдено), подсчёт вхождений числа 40 (два вхождения: на уровнях 2 и 3) и числа 50 (одно вхождение на уровне 0). Затем выполнена инверсия уровня 1: у узлов 30 и 70 поменялись местами поддеревья.

```
> 2
       20
   30
          40
       40
50
       60
   70
       80

> 3
Enter value: 60
Found: 60, level: 2

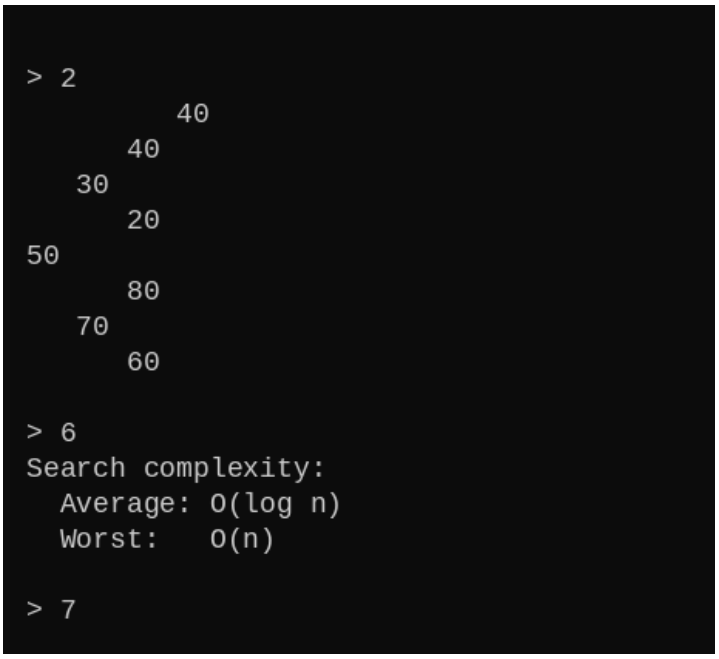
> 3
Enter value: 99
Not found

> 4
Enter value: 40
  found at level 2
  found at level 3
Count: 2

> 4
Enter value: 50
  found at level 0
Count: 1

> 5
Enter level: 1
Level 1 inverted
```

Вывод дерева после инверсии уровня 1 и оценка сложности поиска (пункт меню 6).



Дерево без дубликатов (режим 2). Добавлены числа 50, 30, 70, 30, 20 и 50. Повторные числа 30 и 50 отклонены с сообщением «Duplicate N, skipped», в дерево вошли только четыре различных значения.

```
lab4
1 - tree (with dup)
2 - tree (no dup)
> 2
tree (no dup)


1 - Add
2 - Print
3 - Search
4 - Count
5 - Invert level
6 - Complexity
7 - Exit
> 1
Enter number: 50

> 1
Enter number: 30

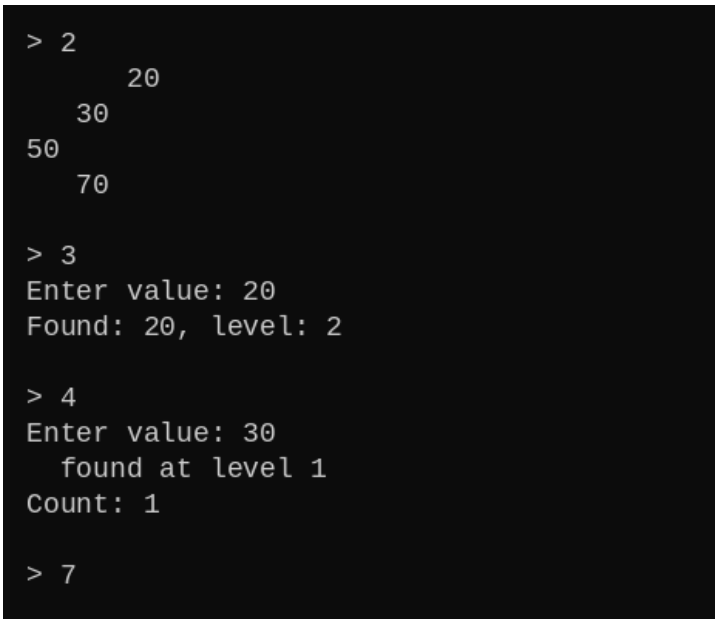
> 1
Enter number: 70

> 1
Enter number: 30
Duplicate 30, skipped

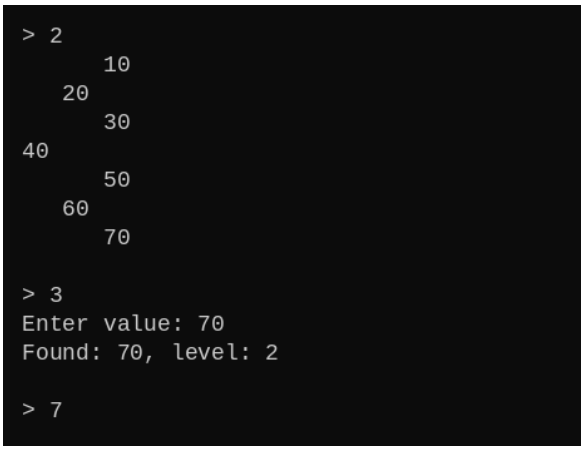
> 1
Enter number: 20

> 1
Enter number: 50
Duplicate 50, skipped
```

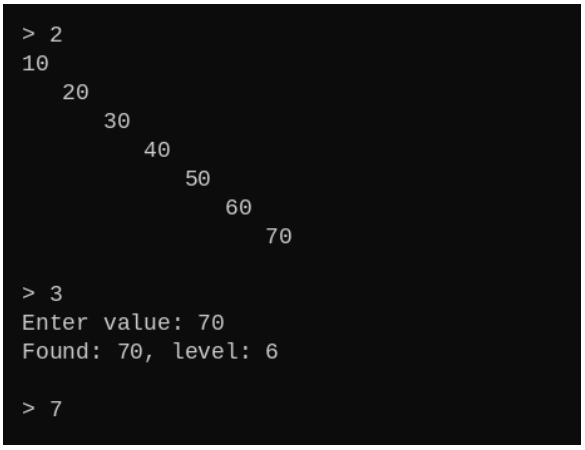
В полученном дереве выполнены поиск значения 20 (уровень 2) и подсчёт вхождений числа 30 (одно вхождение на уровне 1).



Оценка сложности поиска (задание 4). Сбалансированное дерево построено из чисел 40, 20, 60, 10, 30, 50, 70 (высота 2): поиск самого глубокого элемента 70 выполняется на уровне 2, то есть за 3 сравнения.



Вырожденное дерево построено из чисел 10, 20, 30, 40, 50, 60, 70, введённых по возрастанию (высота 6): дерево превратилось в список, и поиск элемента 70 выполняется на уровне 6, то есть за 7 сравнений. Это подтверждает оценки O(log n) и O(n).



Выводы

В ходе выполнения лабораторной работы №4 были изучены:

1. Принципы организации бинарного дерева поиска: структура узла с двумя указателями, правило размещения значений в левом и правом поддеревьях, вывод дерева повёрнутым на 90 градусов.
2. Рекурсивные алгоритмы обработки деревьев: создание, поиск значения с определением уровня, подсчёт числа вхождений, инверсия уровня и освобождение памяти.
3. Поиск и подсчёт вхождений в упорядоченном дереве выполняются за один спуск по одному пути от корня, так как на каждом шаге отбрасывается одно из поддеревьев.
4. Исключение одинаковых элементов достигается проверкой на равенство при спуске по дереву до выделения памяти под новый узел.
5. Сложность поиска по значению равна O(h): в среднем O(log n) для сбалансированного или случайного дерева и O(n) в худшем случае вырожденного дерева, что подтверждено экспериментом на семи элементах (уровни 2 и 6).
6. Работа с динамической памятью: выделение узлов функцией malloc с проверкой результата и освобождение всего дерева функцией free_tree при завершении работы.

Приложение А. Листинг программы

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node *root = NULL;

/* given: create tree */
struct Node *CreateTree(struct Node *root, struct Node *r, int data) {
    if (r == NULL) {
        r = (struct Node *)malloc(sizeof(struct Node));
        if (r == NULL) {
            printf("Memory error\n");
            exit(0);
        }
        r->left = NULL;
        r->right = NULL;
        r->data = data;
        if (root == NULL) return r;
        if (data > root->data) root->left = r;
        else root->right = r;
        return r;
    }
    if (data > r->data)
        CreateTree(r, r->left, data);
    else
        CreateTree(r, r->right, data);
    return root;
}

/* given: print tree (rotated 90 deg) */
void print_tree(struct Node *r, int l) {
    int i;
    if (r == NULL) return;
    print_tree(r->right, l + 1);
    for (i = 0; i < l; i++)
        printf("   ");
    printf("%d\n", r->data);
    print_tree(r->left, l + 1);
}

/* create tree without duplicates */
struct Node *CreateTreeUnique(struct Node *root, struct Node *r, int data) {
    if (r == NULL) {
        r = (struct Node *)malloc(sizeof(struct Node));
        if (r == NULL) {
            printf("Memory error\n");
            exit(0);
        }
        r->left = NULL;
        r->right = NULL;
        r->data = data;
        if (root == NULL) return r;
        if (data > root->data) root->left = r;
        else root->right = r;
        return r;
    }
    if (data == r->data) {
        printf("Duplicate %d, skipped\n", data);
        return root;
    }
    if (data > r->data)
        CreateTreeUnique(r, r->left, data);
    else
```

```
        CreateTreeUnique(r, r->right, data);
    return root;
}

/* search with level */
int search_level(struct Node *r, int data, int level) {
    if (r == NULL) return -1;
    if (data == r->data) return level;
    if (data > r->data) return search_level(r->left, data, level + 1);
    return search_level(r->right, data, level + 1);
}

/* count occurrences with levels */
void count_val(struct Node *r, int data, int level, int *count) {
    if (r == NULL) return;
    if (data == r->data) {
        printf("  found at level %d\n", level);
        (*count)++;
        count_val(r->right, data, level + 1, count);
    } else if (data > r->data) {
        count_val(r->left, data, level + 1, count);
    } else {
        count_val(r->right, data, level + 1, count);
    }
}

/* invert level: swap left/right of all nodes at given level */
void invert_level(struct Node *r, int target, int current) {
    struct Node *tmp;
    if (r == NULL) return;
    if (current == target) {
        tmp = r->left;
        r->left = r->right;
        r->right = tmp;
        return;
    }
    invert_level(r->left, target, current + 1);
    invert_level(r->right, target, current + 1);
}

/* free tree */
void free_tree(struct Node *r) {
    if (r == NULL) return;
    free_tree(r->left);
    free_tree(r->right);
    free(r);
}

/* tree with duplicates */
void task1(void) {
    int cmd, val, lvl, cnt;

    root = NULL;

    printf("tree (with dup)\n\n");

    while (1) {
        printf("\n1 - Add\n2 - Print\n3 - Search\n4 - Count\n5 - Invert level\n6 -
Complexity\n7 - Exit\n> ");
        scanf("%d", &cmd);

        if (cmd == 1) {
            printf("Enter number: ");
            scanf("%d", &val);
            root = CreateTree(root, root, val);
        } else if (cmd == 2) {
            if (root == NULL)
                printf("Tree is empty\n");
            else
```

```
                print_tree(root, 0);
        } else if (cmd == 3) {
            printf("Enter value: ");
            scanf("%d", &val);
            lvl = search_level(root, val, 0);
            if (lvl >= 0)
                printf("Found: %d, level: %d\n", val, lvl);
            else
                printf("Not found\n");
        } else if (cmd == 4) {
            printf("Enter value: ");
            scanf("%d", &val);
            cnt = 0;
            count_val(root, val, 0, &cnt);
            printf("Count: %d\n", cnt);
        } else if (cmd == 5) {
            printf("Enter level: ");
            scanf("%d", &val);
            invert_level(root, val, 0);
            printf("Level %d inverted\n", val);
        } else if (cmd == 6) {
            printf("Search complexity:\n");
            printf("  Average: O(log n)\n");
            printf("  Worst:   O(n)\n");
        } else if (cmd == 7) {
            free_tree(root);
            root = NULL;
            break;
        }
    }
}

/* tree without duplicates */
void task2(void) {
    int cmd, val, lvl, cnt;

    root = NULL;

    printf("tree (no dup)\n\n");

    while (1) {
        printf("\n1 - Add\n2 - Print\n3 - Search\n4 - Count\n5 - Invert level\n6 -
Complexity\n7 - Exit\n> ");
        scanf("%d", &cmd);

        if (cmd == 1) {
            printf("Enter number: ");
            scanf("%d", &val);
            root = CreateTreeUnique(root, root, val);
        } else if (cmd == 2) {
            if (root == NULL)
                printf("Tree is empty\n");
            else
                print_tree(root, 0);
        } else if (cmd == 3) {
            printf("Enter value: ");
            scanf("%d", &val);
            lvl = search_level(root, val, 0);
            if (lvl >= 0)
                printf("Found: %d, level: %d\n", val, lvl);
            else
                printf("Not found\n");
        } else if (cmd == 4) {
            printf("Enter value: ");
            scanf("%d", &val);
            cnt = 0;
            count_val(root, val, 0, &cnt);
            printf("Count: %d\n", cnt);
        } else if (cmd == 5) {
            printf("Enter level: ");
```

```
            scanf("%d", &val);
            invert_level(root, val, 0);
            printf("Level %d inverted\n", val);
        } else if (cmd == 6) {
            printf("Search complexity:\n");
            printf("  Average: O(log n)\n");
            printf("  Worst:   O(n)\n");
        } else if (cmd == 7) {
            free_tree(root);
            root = NULL;
            break;
        }
    }
}

int main(void) {
    int choice;

    printf("lab4\n");
    printf("1 - tree (with dup)\n");
    printf("2 - tree (no dup)\n");
    printf("> ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: task1(); break;
        case 2: task2(); break;
        default: printf("wrong\n"); break;
    }

    return 0;
}
```